MEAL CONNECT

Project Overview & Architecture Notes

JIRA MEAL-197 | Sprint 1: 28 October–3 November 2026 | Version 1.2 (Team Review) | 30 October 2026

Document status: REVIEW DRAFT. This describes the agreed MVP and proposed technical design; it does not claim that the application features have been coded or tested. Approval and final as-built update are pending.

1. Goal and acceptance criterion

MEAL-197 task: “Describe stakeholders, major features, stack, and data flow.” Acceptance criterion: “Documentation reflects the implemented system.” The repository currently has an initial scaffold, while application modules are planned for Sprints 2–7. Reconcile this document with the running code before declaring the final as-built requirement met.

2. Product overview and scope

Meal Connect is a localhost-demonstrated, browser-based platform to connect restaurant/bakery surplus food with verified NGOs and volunteer collectors. A business publishes an available food listing; an NGO selects a quantity from that existing listing and requests it; the owning business fully approves or rejects it; an assigned collector coordinates pickup/delivery; the NGO records distribution. Reports use saved activity. General NGO food-wanted postings and partial approval are not part of the agreed MVP.

Actors and responsibilities

Actor	In-scope responsibility
Restaurant / Bakery	Register/profile; after verification, create/edit/cancel surplus-food listing and approve or reject NGO quantity requests.
Verified NGO / Charity	Find/filter available listings; request selected quantities; track collection; record distribution and beneficiary counts.
Volunteer / Collector	Access assigned pickups and update pickup and delivery statuses.
Administrator	Verify organizations; manage users/listings/requests; moderate questionable records; view basic impact reports.
Beneficiaries	Receive food through NGOs; no login/account in this MVP.

Included / excluded boundaries

Included: registration and login, hashed passwords, roles and organization verification; donations and search; NGO requests; full approval/rejection with safe stock allocation; pickup and distribution; in-app notifications; administration; basic persisted-data reporting; mobile-responsive browser interface.

Excluded: native mobile apps, payments/fundraising, AI matching/prediction, live GPS or delivery-provider integration, QR pickup confirmation and production cloud deployment. Food-safety certification is not promised by the system.

3. Architecture: logical three-tier design

Presentation (Angular 21 SPA with TypeScript, components, router, forms and HttpClient) → application (Python FastAPI REST endpoints, authorization, validation and services) → persistence (PostgreSQL; SQLAlchemy/Alembic implementation choices). Three-tier boundaries remain unchanged; no Docker in the localhost MVP.

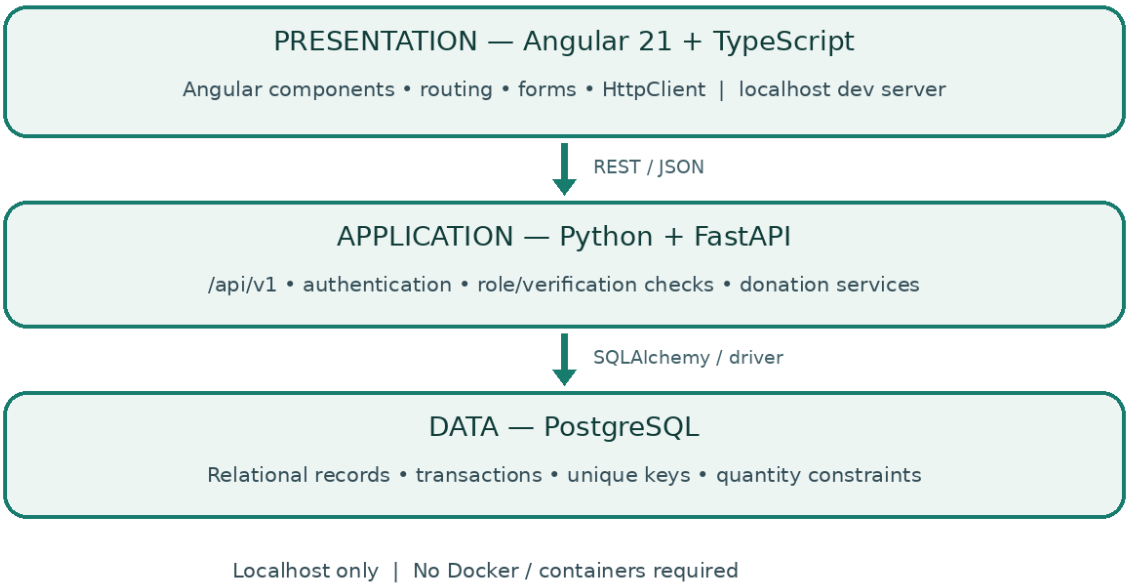


Figure 1. High-level logical architecture (MEAL-61; proposed, not a deployed network diagram).

Component responsibilities

Layer / module	Boundary / responsibility
Frontend	Angular 21 SPA: TypeScript components, routing, typed forms, HttpClient services; UI only (no direct database credentials or authorization decisions).
FastAPI routes	Versioned HTTP/JSON interface, request parsing, auth dependencies and response codes.
Domain services	Organization verification, donation availability, NGO request decisions, collector status changes, distribution confirmation, notifications and report rules.
Data access / PostgreSQL	Entities, foreign keys, unique/check constraints, migrations, audit timestamps, concurrency-safe stock transactions.

4. Core workflow and quantity rules

1. Business organization is approved; authorized staff publish an available donation with quantity, safety disclosures, pickup address and deadline.
2. Verified NGO browses a listing and requests any valid quantity not exceeding current available quantity. This is a request against an existing donation, not a separate general demand posting.
3. Owning business approves ALL requested_quantity or rejects the request. No approved_quantity column or partial approval is needed.
4. Upon approval, reserve exactly requested_quantity, under a single transaction with row locking/atomic conditional update, preventing over-allocation during simultaneous decisions.
5. Approved request is assigned to a collector; pickup/delivery progress is recorded; NGO records final distribution; reports use actual persisted completion records.

Example

Event	Quantity (portions)
Restaurant publishes	100
NGO requests	30
Business approves the full request	30
donations.allocated_quantity increase	30
Remaining available	70

Do not accept requests for expired/cancelled donations. An approved cancellation before collection may release allocation once, using the same transaction; release after collection needs an explicitly approved reversal policy. These controls are design rules, not yet implemented.

5. Data model and logical relationships

The reviewed MEAL-6 baseline contains ten tables: users, organizations, organization_memberships, organization_verifications, donations, donation_requests, pickups, distribution_records, notifications and status_history. The DBML source is the schema-design authority; this document is only an overview.

Relationship / condition	Intended rule
users ↔ organizations	Many-to-many through organization_memberships, unique organization/user pair.
organization → donations	A verified business organization owns many donations; individual creators recorded.
donation → donation_requests	One donation has many requests from verified NGOs.
request → pickup	Zero or one pickup per request (UNIQUE pickups.request_id).
pickup → distribution_records	Zero or one final distribution per pickup (UNIQUE distribution_records.pickup_id).
quantity / history	allocated_quantity between zero and total_quantity; transactions and status_history trace approvals.

Database validation beyond simple table-local constraints (for example membership authorization, allowed status transitions and cross-table distribution amount) must be implemented by backend service logic and/or database triggers in later sprints.

6. API conventions and integration guidance

Topic	Current proposed convention
Base path	/api/v1; localhost example http://localhost:8000/api/v1
Data	JSON UTF-8; snake_case properties; timezone-aware ISO 8601 timestamps
Resource naming	Lowercase plural paths, IDs as path variables
Methods	GET read; POST create/action; PATCH partial change; DELETE only where approved
Auth	Server-side role, active membership, organization type/verification and ownership checks; JWT proposed, not selected
Success	200/201/204 as appropriate
Errors	400, 401, 403, 404, 409, 422, 500; consistent safe machine-readable error envelope proposed
API documentation	FastAPI OpenAPI /docs once backend is implemented

Illustrative (not implemented): GET /api/v1/donations; POST /api/v1/donations; POST /api/v1/donations/{donation_id}/requests; POST /api/v1/requests/{request_id}/approve; PATCH /api/v1/pickups/{pickup_id}; GET /api/v1/reports/impact. Final route names and error shape must follow the MEAL-62 review decision.

7. Development environment and repository

Development target: Windows localhost without Docker, per MEAL-63 and MEAL-163 v1.2. All seven members report Angular 21, Python and PostgreSQL installed; exact Node.js/npm/Angular CLI, Python, PostgreSQL and dependency versions still need team verification. Angular CLI dev server runs separately from FastAPI/Uvicorn and the local PostgreSQL service. Configure CORS for the actual frontend origin and do not commit credentials.

MEAL-161 GitHub repository: https://github.com/rivipahanviksura/meal-connect (private). Existing scaffold: frontend/, backend/app/{api,models,schemas,services}, database/, docs/; Angular CLI will establish frontend/src/app, angular.json, package.json and package-lock.json in Sprint 2 (not yet verified as committed). Use feature branches into develop; main remains stable.

Suggested additions, not yet verified as committed: Angular src/app/core, shared and features modules; backend/app/core, backend/app/db, backend/app/security, backend/alembic, backend/requirements.txt, sanitized .env.example. Do not treat this layout as already implemented.

8. Sprint mapping and handoff

Sprint	Expected milestone / status
1 · 28 Oct–3 Nov	Requirements, ERD, architecture, user journeys, wireframes, repository scaffold; documents drafted for review.
2 · 4–10 Nov	Local application setup, PostgreSQL, authentication, roles, organization verification.
3 · 11–17 Nov	Donation create/edit/cancel; browse, search, filtering.
4 · 18–24 Nov	NGO quantity requests and full approve/reject with allocation checks.
5 · 25 Nov–1 Dec	Pickup and NGO distribution workflow.
6 · 2–8 Dec	Admin, basic in-app notifications, recorded-data reporting.
7 · 9–15 Dec	Integration, security, QA, documentation.
Final · 16–18 Dec	Final verification, demo and submission.

9. Decisions, risks and as-built verification

Item	Status / owner action
Full approve or reject; no general NGO request posting	Agreed MVP business rule; communicate to all feature owners.
Database schema and migration constraints	MEAL-6 design documented; implement and test in Sprint 2.
Auth mechanism and token expiry	Proposed JWT; backend/team must select and record final contract.
Exact dependency pins, module paths, status enums	To confirm alongside MEAL-63 v1.2/163 and ERD/API owners.
Concurrent approval, expiry, cancellation	Critical implementation/testing tasks; do not treat design notes as tested software.
Architecture/API and supervisor sign-off	Pending review; update diagrams and this note on decisions.
As-built documentation reflection	Pending actual code and local running-app verification; revisit before final Done.

Revision v1.2 decision: Team selected Angular 21 + TypeScript for frontend, with FastAPI and PostgreSQL running directly on localhost (Docker excluded). This is a technology/tooling decision only; MVP features remain unchanged. MEAL-5, MEAL-61, MEAL-62, MEAL-63 and MEAL-163 v1.2 are the updated interface and environment references.

Review checklist — MEAL-197

☐ Stakeholders and MVP features match approved scope

☐ Architecture diagram and module boundaries match MEAL-5 and MEAL-61

☐ Entity references/quantity rules match MEAL-6 DBML and decision note

☐ API examples and error conventions match MEAL-62

☐ Repository layout and version assumptions checked with actual GitHub and MEAL-63/163

☐ Running implementation checked and any differences recorded

☐ Team/supervisor review and sign-off recorded

Reviewer: _____ Date: _____ Decision: Approved / Revisions required

10. Source documents

Meal Connect Project Proposal; MVP Scope v1.1 & Prioritized Requirements; MEAL-5 and MEAL-61 finalized Angular 21 / No-Docker v1.2; MEAL-62 API v1.2; MEAL-63 and MEAL-163 v1.2; MEAL-6 reviewed DBML / business decision; MEAL-161 GitHub structure; MEAL-209 Risk & Decision Log v1.2.